

PRODUCT REQUIREMENTS DOCUMENT

PlacePro

Smart Placement Management System

Version 1.0

Date: July 6, 2026

Owner: PlacePro Project Team

Status: Active Development

Confidential — Internal / Academic Use Only

Contents

- 1.Product Overview
- 2.Objectives and Goals
- 3.Target Users
- 4.Problem Statement
- 5.Proposed Solution
- 6.Core Features
- 7.System Workflows
- 8.Data Flow Overview
- 9.Functional Requirements
- 10.Non-Functional Requirements
- 11.Technology Stack
- 12.System Architecture
- 13.Database Design
- 14.Risk Assessment
- 15.Future Enhancements

1. Product Overview

1.1 Product Vision

PlacePro is a desktop-based placement management system that gives a college Training and Placement Office (TPO) a single, reliable platform for running the entire campus placement lifecycle — from student registration and company onboarding through drive publication, application tracking, interview coordination, and final reporting. It is built around the reality of most placement cells today: important information is scattered across spreadsheets, email threads, and paper registers, which slows communication, hides mistakes, and leaves students guessing about where they stand.

The long-term vision behind PlacePro is a system that removes manual record-keeping from the placement office, gives students transparent and timely visibility into drives and application outcomes, and gives administrators the data they need to make decisions and satisfy institutional reporting requirements — while remaining simple enough to install and run on ordinary college lab machines without any cloud dependency.

1.2 Description

PlacePro is a full-featured desktop application built with Java and Java Swing for the interface, JDBC for database connectivity, and MySQL as the persistent data store. It provides role-based access for four kinds of users — students, placement officers, administrators, and recruiters — each working through an interface tailored to their responsibilities. Placement officers and administrators maintain student records, onboard companies, publish drives, and manage the application and interview pipeline; students discover eligible drives, apply, and track their own progress; recruiters log in through a restricted portal to review shortlists and record interview outcomes. Source code is version-controlled with Git and hosted on GitHub.

1.3 Product Snapshot

Field	Detail
Application Type	Desktop application (Java Swing)
Database	MySQL, accessed via JDBC
Target Institution	Engineering and degree colleges with a dedicated placement cell
Primary Users	Students, Placement Officers, Administrators, Recruiters
Deployment Model	Local installation on college lab or office machines, connected to a shared MySQL instance on the campus LAN
Version Control	Git, hosted on GitHub

1.4 One-Line Description

PlacePro is a Java Swing desktop application that centralizes campus placement operations — student profiles, company drives, applications, interviews, and reports — into one MySQL-backed system for colleges and placement coordinators.

2. Objectives and Goals

2.1 Product Goals

- Centralize placement data — consolidate student records, company information, drive schedules, and application statuses into a single MySQL database accessible through one desktop application.
- Automate manual workflows — replace spreadsheet-based tracking with structured forms, validation, and automated status updates for applications and interview rounds.
- Enable role-based access — provide distinct interfaces and permissions for students, placement officers, recruiters, and administrators, so that data integrity and appropriate visibility are preserved.
- Improve transparency for students — let students view published drives, check eligibility, submit applications, and monitor status without depending on email or notice-board updates.
- Support reporting and auditability — generate placement summaries, drive-wise statistics, and outcome reports usable for internal review and accreditation documentation.
- Demonstrate sound engineering practice — deliver a well-layered Java desktop application using JDBC and Git-based version control, suitable for technical evaluation.

2.2 Business Goals

- Reduce the administrative overhead placement officers spend on manual data entry and status reconciliation across multiple tools.
- Increase placement process efficiency by giving every stakeholder a single source of truth for drives, shortlists, and schedules.
- Improve data accuracy by eliminating the inconsistencies that come from disconnected spreadsheets during high-volume placement seasons.
- Enhance the student experience with timely, self-service access to opportunities and application progress.
- Enable institutional reporting with exportable summaries required for management reviews and accreditation.
- Establish a reusable, maintainable codebase that future batches can extend.

2.3 Success Criteria

ID	Criterion	Measurement
SC01	Functional completeness	All core modules — authentication, student management, company/drive management, applications, and reporting — are implemented and demonstrable end to end.
SC02	Data persistence	All records are stored and retrieved correctly from MySQL via JDBC with no data loss across application restarts.
SC03	Role-based access	Student, officer, recruiter, and administrator roles each enforce correct permissions; unauthorized actions are blocked at the application layer.
SC04	Usability	A new user can complete their primary task — apply to a drive, or create a drive — within five minutes without external documentation.
SC05	Performance	Standard CRUD operations and report generation complete within three seconds on a typical lab machine with up to 500 student records.
SC06	Reliability	The application handles invalid input, connection failures, and duplicate submissions gracefully with user-friendly messages.
SC07	Code quality	Source code follows a layered structure (UI, service, DAO), is version-controlled on GitHub, and carries a meaningful commit history.
SC08	Academic deliverables	PRD, working application, database schema, and demo readiness are completed within the project timeline.

3. Target Users

PlacePro serves four distinct groups within the campus placement ecosystem, each with its own responsibilities, goals, and pain points. Understanding these personas is what shapes the role-based structure of the application described in later sections.

3.1 Student — Priya Sharma, Final-Year CSE Student

Students are registered final-year candidates who need a single, trustworthy place to discover opportunities, understand whether they qualify, and follow their own progress. Today, drive announcements are scattered across notice boards, email, and messaging groups, so students miss deadlines, waste effort applying to drives they are not eligible for, and have no self-service way to check status — they must ask officers or classmates and often wait anxiously during peak season. PlacePro answers this with a student dashboard that lists active drives filtered by eligibility, shows company, role, package, and deadline details in one structured view, allows one-click application with instant confirmation, and keeps a personal history of every application a student has ever made.

3.2 Placement Officer — Mr. Rajesh Kumar, Head, Training and Placement Cell

Placement officers run the day-to-day placement process: registering students, onboarding companies, publishing drives,

filtering eligible applicants, coordinating interviews, and producing reports. Their present reality is dominated by manual cross-referencing across Excel files, error-prone eligibility checks performed by hand for every drive, constant interruptions from students asking for status updates, and days of effort spent compiling reports at the end of the season. PlacePro centralizes student, company, drive, and application data in one database, automates eligibility filtering against rules defined at drive creation, and produces placement statistics on demand rather than through manual aggregation — freeing officers to focus on coordination rather than data entry.

3.3 Recruiter — Ms. Ananya Desai, Talent Acquisition Lead, TechCorp Solutions

Recruiters are the hiring representatives who visit campus or coordinate remotely to interview shortlisted students. They currently receive shortlists in inconsistent formats, struggle to estimate the eligible candidate pool before committing to a visit, and exchange scheduling details through lengthy email threads, all of which can make a college placement cell feel disorganized. PlacePro gives recruiters a dedicated, restricted login that shows only the shortlists and schedules relevant to their own company, so they can confirm interview details and record round-wise outcomes directly in the system instead of over email — improving both the candidate experience and the college's standing as a placement partner.

3.4 Administrator — Dr. Sunita Menon, Dean of Student Affairs / IT Liaison

Administrators are responsible for platform governance: provisioning accounts, configuring system settings, protecting student data, and reviewing aggregated reports before they go to institutional leadership. Without a central system, this oversight is close to impossible — placement data lives on individual laptops with no formal backup, there is no way to audit who changed a shortlist or when, and report accuracy cannot be verified when the underlying sources are fragmented spreadsheets. PlacePro enforces role-based authentication so each group only accesses what its role permits, stores everything in a managed MySQL database with standard backup practices, and produces reports directly from the authoritative dataset rather than from files assembled by hand.

4. Problem Statement

4.1 Core Problem

College placement offices lack a unified, reliable system to manage the full placement lifecycle. Student records, company drives, applications, and interview outcomes are scattered across spreadsheets, email chains, and physical registers, creating delays, data errors, and poor visibility for students, placement officers, and recruiting companies alike. Despite how much campus placements matter to institutional reputation and student outcomes, most colleges still coordinate this high-stakes, multi-stakeholder process with manual tools that were never designed for it.

4.2 Current Problems by Stakeholder

Stakeholder	Key Problems
Students	No single source of truth for available drives, eligibility criteria, or deadlines; status updates depend on notice boards or informal messaging; missed opportunities and difficulty tracking application history across companies.
Placement Officers	Heavy reliance on Excel sheets and paper forms; duplicate data entry across files; no real-time dashboard for drive participation or statistics; high risk of data loss, version conflicts, and transcription errors during peak season.
Recruiters	Shortlists arrive in inconsistent formats with varying fields; limited visibility into the eligible pool before a campus visit; scheduling requires repeated back-and-forth; delayed feedback loops affect offer processing.

4.3 The Existing Manual Process

Without dedicated software, the typical placement workflow follows a repetitive, spreadsheet-driven cycle that repeats for every company drive throughout the season: officers collect and consolidate student profiles into a master workbook; company details arrive by phone or email and are recorded in separate files; eligible students are filtered manually from the master sheet and notices are posted on boards or sent by bulk email; a drive-specific spreadsheet tracks applicants while shortlists are shared with recruiters in non-standard formats; interview schedules are prepared separately and communicated verbally or through class representatives; selection outcomes are collected after the fact and entered into

result sheets, often days later; and, finally, end-of-season statistics are compiled by manually aggregating numbers across many files. Each cycle multiplies the administrative effort and the risk of error.

4.4 Why Current Methods Fall Short

Spreadsheets, generic project-management tools, and ad-hoc communication channels are not purpose-built for placement workflows: they lack enforced data models, role-based access, validation rules, and defined workflow states such as applied, shortlisted, interviewed, and selected. Updates made in one officer's local file are invisible to everyone else until manually shared, and manual tools degrade quickly at the scale of thirty to fifty drives and thousands of applications per season. There is no way to guarantee that students only see their own applications, no foreign-key integrity linking students, companies, drives, and outcomes, and enterprise applicant-tracking software is both financially and operationally out of reach for a college lab environment. Meanwhile, students who are used to self-service digital portals for admissions and examinations reasonably expect the same standard for placements. The gap is not a lack of effort by placement staff — it is the absence of a structured, role-aware, database-backed system built specifically for this domain.

5. Proposed Solution

PlacePro addresses the problems identified above through a dedicated desktop application that centralizes placement operations into a single MySQL-backed system, with role-based Java Swing interfaces for students, placement officers, administrators, and recruiters. The system is organized into three logical layers.

5.1 Solution Architecture

The data persistence layer is a normalized MySQL database that stores every entity — students, companies, drives, applications, interview schedules, and placement outcomes — with relational integrity enforced through primary and foreign keys, and JDBC providing type-safe connectivity between the application and the database.

The business logic and data access layer encapsulates validation rules, eligibility checks, application state transitions, and report generation in a service layer, with a JDBC-based DAO layer handling all MySQL persistence on its behalf. Placement officers and administrators interact with management modules, while students interact with a restricted student portal — both backed by the same consistent data model.

The presentation layer is a Java Swing desktop interface that delivers form-based data entry, searchable tables, status dashboards, and exportable reports, designed for placement-cell workstations and student lab machines on a local network, with no internet dependency for core operations.

5.2 How PlacePro Resolves Each Problem Area

Problem Area	PlacePro Resolution
Fragmented records	A single MySQL database acts as the authoritative source for all placement data.
Manual eligibility filtering	Drive configuration stores eligibility rules; the system filters eligible students automatically.
Delayed student updates	Students log in to view drive listings, submit applications, and track status in real time.
Shortlist management	Officers review applications within the app, update statuses, and generate standardized shortlists.
Interview coordination	Interview rounds and outcomes are recorded directly against applications in the system.
End-of-season reporting	Built-in reports aggregate placement statistics without manual spreadsheet work.
Access control	Role-based login ensures each user sees only the data and actions appropriate to their role.
Data safety	Persistent MySQL storage with standard backup practices replaces fragile single-file dependencies.

5.3 Why This Approach Works

The approach is purpose-built for campus placement workflows rather than adapted from generic tools, and carries a low deployment barrier — it runs on existing college lab machines with a local or network MySQL instance and needs no cloud subscription. The stack is familiar and well-documented, appropriate for both academic evaluation and long-term maintenance, and delivers immediate operational value: officers gain a working admin console from day one, and students

gain self-service access from day one. Its layered, extensible foundation allows future enhancements without redesigning the core data model.

5.4 Scope of the Project

In Scope

Area	Included
Platform	Desktop application built with Java and Java Swing
Database	MySQL database with JDBC connectivity
User roles	Student, Placement Officer, Administrator, Recruiter
Notifications	In-app notifications; external email or SMS delivery excluded
Student management	Registration, profile maintenance, eligibility criteria
Company management	Company profiles, job roles, package details
Drive management	Create, publish, update, and close placement drives
Applications	Student apply, officer/admin review, shortlist/reject workflow
Interview tracking	Schedule rounds, record outcomes (selected / rejected / on hold)
Authentication	Login, session management, role-based navigation
Reporting	Placement summaries, drive-wise and department-wise statistics

Out of Scope

Area	Excluded	Rationale
Web or mobile clients	Browser-based or Android/iOS apps	Scope is limited to a Java Swing desktop application
Cloud deployment	AWS, Azure, or SaaS hosting	Local desktop plus local/network MySQL only
Online payments	Registration fees, company payments	Not required for the core placement workflow
Video interviews	Zoom, Teams, or WebRTC integration	External tooling; not part of desktop scope
AI resume matching	Automated skill-to-job matching	Advanced feature deferred to a future version
Multi-college federation	Cross-institution placement portals	Single-institution scope for v1.0
External notifications	Automated email or SMS delivery	In-app notifications are in scope; external delivery is deferred
Social login	Google or LinkedIn authentication	Local credential-based authentication only

These boundaries assume a MySQL server is installed and reachable on the deployment machine or campus LAN, that administrators are responsible for the accuracy of company and drive data they enter, that students receive their login credentials from the placement cell upon registration, and that the application runs on machines with Java Runtime Environment 11 or higher.

6. Core Features

PlacePro's feature set is organized around five functional clusters that together cover authentication, student self-service, day-to-day placement operations, the recruiter experience, and institutional reporting. Each cluster is implemented as a cohesive module rather than a collection of unrelated screens, so that the underlying data model — students, companies, drives, applications, and interviews — stays consistent no matter which role is interacting with it.

6.1 Authentication and Role-Based Access

Every user of PlacePro enters through a role-specific login: students, placement officers, administrators, and recruiters each authenticate against credentials stored in MySQL and are routed to an interface that exposes only the modules appropriate to their role. Administrators can create, deactivate, and reset accounts for every role; placement officers are granted access to company, drive, application, and interview modules but cannot touch system-level administrator settings; and recruiters are limited strictly to the drives and shortlists tied to their own company. A consistent logout action is available from every interface, terminating the session and clearing role-specific state — a small but important safeguard on shared lab machines where the next student or officer to sit down should never inherit someone else's session.

6.2 Student Self-Service

Students register with validated personal and academic details — name, roll number, branch, CGPA, and contact information — with duplicate roll numbers or email addresses rejected outright. Once logged in, a dashboard summarizes active drives, pending and shortlisted applications, and upcoming deadlines, with quick navigation to drive listings, application tracking, and resume management. Students can upload a resume in PDF or DOC format, which the system validates for type and size and links to their profile so officers can review it during shortlisting. Before a student can apply to a drive, an eligibility checker automatically evaluates their academic profile against the drive's configured rules — minimum CGPA, allowed branches, and maximum backlogs — and reports a clear eligible or ineligible result with specific reasons, preventing wasted applications and unnecessary officer workload. Eligible students can then submit an application with one click; the system timestamps it, blocks duplicate submissions to the same drive, and immediately confirms the reference number. From that point on, application tracking shows the full status history — Applied, Shortlisted, Interview Scheduled, Selected, Rejected, or On Hold — updated in real time as officers and recruiters progress the record, removing the need to visit the placement office for a status check.

6.3 Placement Operations Management

Placement officers and administrators maintain the company directory — name, industry, contact details, and the job roles and package bands each company offers — and can search, edit, or deactivate entries as relationships evolve. From a registered company, officers create placement drives with defined eligibility rules and a lifecycle that moves through draft, published, closed, and completed states; publishing a drive immediately makes it visible to eligible students, while closing it stops further applications once a deadline passes. As applications arrive, officers review student profiles and resumes directly within the application, update status through the same workflow students see, and record a timestamped history of every transition so that shortlist decisions remain auditable. Interview scheduling lets officers assign date, time, venue, and round type to each shortlisted application, supports multiple rounds — aptitude, technical, HR — with round-wise outcomes, and gives officers a schedule view filterable by drive, date, or company for coordinating on-campus visits. A student-search tool with filters for name, roll number, branch, CGPA range, and placement status, alongside a matching company-search tool, replaces the experience of scrolling through master Excel sheets to find a single record.

6.4 Recruiter Portal

Recruiters authenticate through a restricted portal scoped entirely to their own company: they can view the shortlist an officer has published for a drive, confirm the proposed interview schedule, and record round-wise outcomes — selected, rejected, or on hold — that flow straight back into the application's status history. Because visibility is limited to company-specific data, a recruiter never sees unrelated student records or another company's drives, and the college no longer needs to exchange shortlists or schedules through unstructured email threads.

6.5 Notifications, Reporting, and Analytics

An in-app notification inbox alerts each user — by role — to the events that matter to them: a newly published drive, a status change, or a scheduled interview, with unread counts visible from the dashboard. On the reporting side, officers and administrators can generate placement summaries filtered by academic year, department, company, or date range, and export them as PDF or CSV directly from the desktop client, replacing the manual spreadsheet consolidation that historically ate days of effort at season's end. An analytics dashboard complements these static reports with live visual summaries — total placements, placement percentage by department, average package, top recruiting companies, and application-to-selection conversion rates — computed dynamically from the current MySQL data so that leadership can monitor the season's progress without waiting for a custom report.

7. System Workflows

The two workflows below trace how the feature clusters in Section 6 come together end to end — first from a student's point of view applying to a single drive, and then from the administrative side of setting up a drive and carrying it through to a recruiter's final decision.

7.1 Primary Flow — Student Placement Journey 7.2 Secondary Flow — Administrative Setup

A student registers, applies to a published drive, is shortlisted, receives an interview schedule, and is notified An administrator onboards a company and creates a drive; an officer manages incoming applications; a

of the outcome. The system checks eligibility recruiter confirms schedules and records outcomes automatically before any application is accepted. once on-campus rounds are complete.

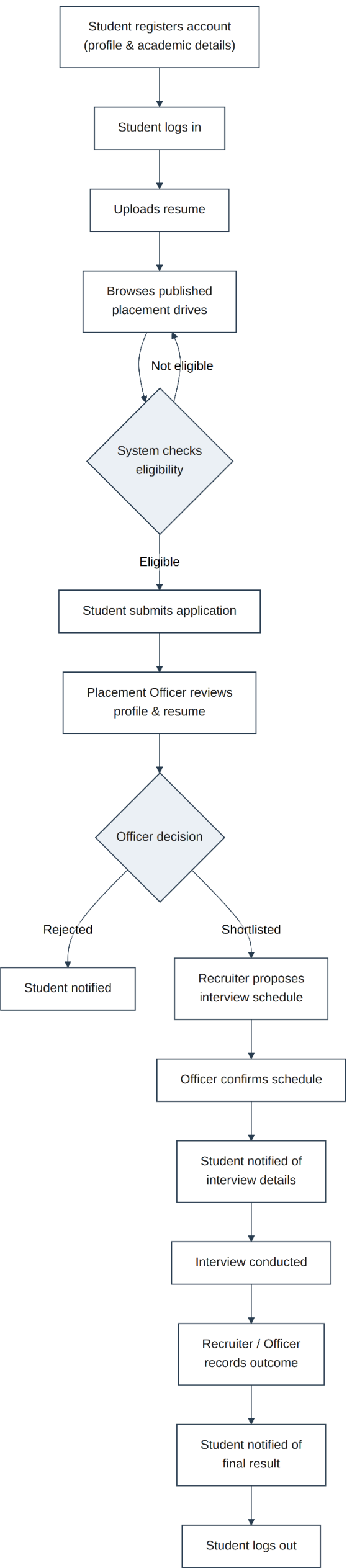


Figure 1—Primary flow: student placement journey.

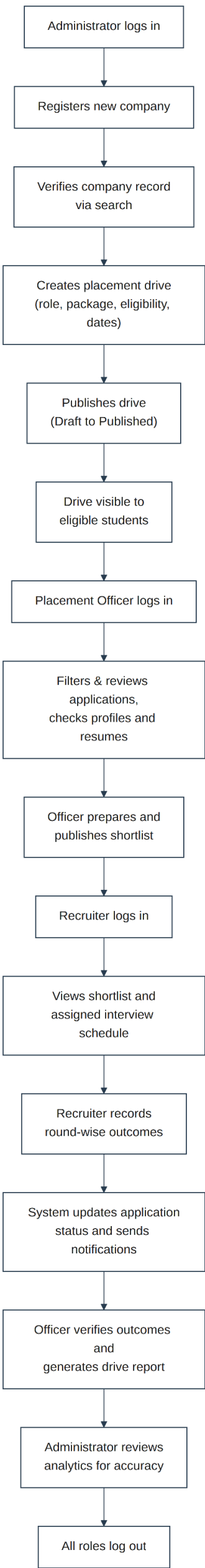


Figure 2—Secondary flow: drive setup and recruiter coordination.

8. Data Flow Overview

The level-1 data flow diagram below abstracts away user interface detail and shows how information actually moves through PlacePro: the four external entities on the left and right, the six core processes that transform their requests, and the five persistent data stores each process reads from or writes to. It is a useful complement to the use-flow diagrams in Section 7, because it makes explicit which data store backs each interaction—for example, that an application can only be accepted after the application process consults both the student/resume store and the drive-rules store to confirm eligibility.

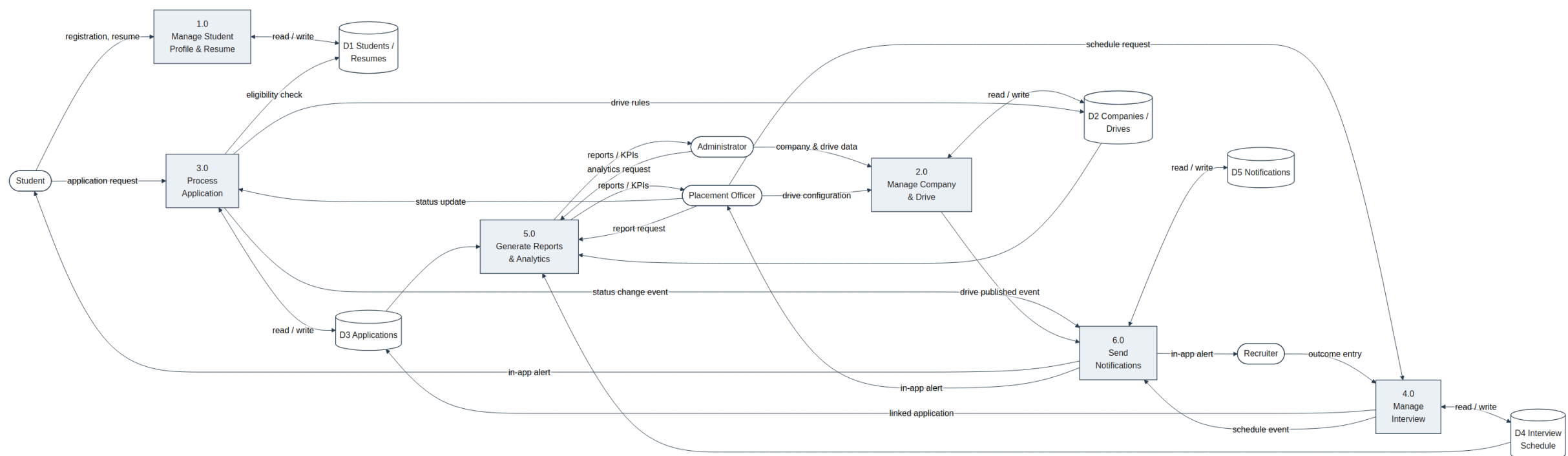


Figure 3—Level-1 data flow diagram for PlacePro.

Two aspects of this diagram are worth calling out. First, every write to the notifications store is triggered by an event in another process — a published drive, a status change, or a new schedule — rather than by a user directly, which keeps the notification rules centralized in one place instead of duplicated across the application. Second, the reporting process only ever reads from the other four stores; it has no write path of its own, which guarantees that generated reports can never silently alter the underlying placement records they summarize.

9. Functional Requirements

The functional requirements below define the mandatory behaviors PlacePro must exhibit. Each is phrased as a shall-statement and assigned one of three priorities: High requirements are core functionality without which the primary use flows cannot operate; Medium requirements are important supporting functionality whose absence degrades the experience but leaves the system usable; Low requirements are desirable enhancements that may be deferred without blocking a placement season.

9.1 Authentication and Account Management

ID	Requirement
FR-01	The system shall allow new students to register with name, roll number, branch, CGPA, email, contact number, and password. (High)
FR-02	The system shall reject registration when the roll number or email address already exists. (High)
FR-03	The system shall validate all mandatory registration fields and display descriptive errors before persisting a record. (High)
FR-04	The system shall authenticate students against MySQL-stored credentials and establish a student-role session on success. (High)
FR-05	The system shall redirect authenticated students to the Student Dashboard and deny access to administrator and officer modules. (High)
FR-06	The system shall authenticate administrator credentials and grant access to user management, configuration, analytics, and reporting. (High)
FR-07	The system shall allow administrators to create, deactivate, and reset accounts for student, officer, and recruiter roles. (High)
FR-08	The system shall authenticate officer credentials and grant access to company, drive, application, and interview modules. (High)
FR-09	The system shall prevent officers from modifying system-level administrator settings. (Medium)
FR-10	The system shall authenticate recruiter credentials and restrict visibility to the recruiter's assigned company. (Medium)
FR-11	The system shall allow recruiters to view shortlisted candidates and interview schedules for assigned drives only. (Medium)
FR-38	The system shall terminate the active session, clear client state, and redirect to the login screen on logout. (High)
FR-39	The system shall provide a logout action from every role-based interface. (High)

9.2 Company and Drive Management

ID	Requirement
FR-12	The system shall allow authorized users to create, edit, and deactivate company profiles with name, industry, contact person, and contact details. (High)
FR-13	The system shall persist company records and associate each with its drives and historical placement outcomes. (High)
FR-14	The system shall allow authorized users to create drives linked to a company, job role, package, eligibility criteria, visit date, and deadline. (High)
FR-15	The system shall support drive lifecycle states — Draft, Published, Closed, Completed — and enforce valid transitions. (High)
FR-16	The system shall display published drives to eligible students only and block applications to closed or draft drives. (High)
FR-37	The system shall allow officers and administrators to search companies by name, industry, and active drive status. (Medium)

9.3 Student Applications and Eligibility

ID	Requirement
FR-17	The system shall display active drives, application counts by status, and upcoming deadlines on the Student Dashboard after login. (High)
FR-18	The system shall provide navigation shortcuts from the dashboard to drives, application tracking, profile, and resume upload. (Medium)
FR-19	The system shall allow students to upload resumes in PDF or DOC format and validate type and size before acceptance. (High)
FR-20	The system shall store the resume file reference and associate it with the student profile and future applications. (High)
FR-21	The system shall evaluate eligibility against a drive's rules — minimum CGPA, allowed branches, maximum backlogs — and return an eligible or ineligible result. (High)
FR-22	The system shall display specific ineligibility reasons when a student fails validation for a selected drive. (Medium)
FR-23	The system shall allow eligible students to submit an application, recorded with status Applied and a submission timestamp. (High)
FR-24	The system shall prevent a student from submitting more than one application to the same drive. (High)
FR-36	The system shall allow officers and administrators to search students by name, roll number, branch, CGPA range, and placement status. (Medium)

9.4 Application Tracking and Interviews

ID	Requirement
FR-25	The system shall maintain application records with statuses Applied, Shortlisted, Interview Scheduled, Selected, Rejected, and On Hold. (High)
FR-26	The system shall allow officers to update application status, reflected immediately in the student's tracking view. (High)
FR-27	The system shall record a timestamped status history for each application state transition. (Medium)
FR-28	The system shall allow officers and recruiters to schedule interview rounds with date, time, venue, and round type. (High)
FR-29	The system shall support multiple interview rounds per application with round-wise outcomes. (Medium)

9.5 Notifications, Reporting, and Analytics

ID	Requirement
FR-30	The system shall generate in-app notifications when a drive is published, a status changes, or an interview is scheduled. (Medium)
FR-31	The system shall maintain a per-user notification inbox with read/unread indicators and an unread count. (Medium)

FR-32	The system shall generate placement reports filterable by academic year, department, company, and drive. (High)
FR-33	The system shall export generated reports as PDF or CSV from the desktop client. (Medium)
FR-34	The system shall display visual analytics including total placements, department-wise percentage, average package, and conversion rate. (Medium)
FR-35	The system shall compute analytics dynamically from live MySQL data without manual export. (Medium)

In total, PlacePro defines thirty-nine functional requirements (FR-01 through FR-39): twenty-five rated High, fourteen rated Medium, and none deferred as Low — reflecting that every requirement in this document is considered necessary for a demonstrable v1.0 release.

10. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints PlacePro must satisfy alongside its functional behavior. Thirty-four requirements (NFR-01 through NFR-34) are grouped into thirteen categories, summarized below with representative requirements from each.

Category	Representative Requirement	Count
Performance	Standard CRUD operations complete within 3 seconds with up to 500 student records; reports within 5 seconds.	3 (NFR-01– 03)
Security	Passwords hashed with BCrypt (work factor ≥ 10); failed logins throttled after 5 attempts in 10 minutes.	4 (NFR-04– 07)
Reliability	Multi-step writes use transactional JDBC operations to ensure atomicity.	2 (NFR-08– 09)
Availability	The application sustains 8+ hours of continuous operation without restart under expected load.	2 (NFR-10– 11)
Usability	A first-time student completes registration through submission within 5 minutes unaided.	3 (NFR-12– 14)
Maintainability	Codebase follows a layered architecture (UI $\rightarrow$ Service $\rightarrow$ DAO) and is version-controlled on GitHub.	3 (NFR-15– 17)
Scalability	The system supports 500 students and 50 active drives without CRUD degradation.	2 (NFR-18– 19)
Portability	The application runs on Windows, macOS, and Linux with JRE 11+.	2 (NFR-20– 21)
Database	All SQL queries use parameterized Prepared Statements to prevent injection.	3 (NFR-22– 24)
Backup	Documentation includes mysqldump backup procedures executed before each placement season.	2 (NFR-25– 26)
Logging	The application logs authentication events, database errors, and unhandled exceptions with timestamp and severity.	2 (NFR-27– 28)
Error Handling	SQLExceptions are caught at the DAO layer; the UI shows only user-friendly messages.	3 (NFR-29– 31)
Response Time	The Student Dashboard loads within 2 seconds after login; eligibility checks return within 1 second.	3 (NFR-32– 34)

11. Technology Stack

PlacePro is built as a standalone desktop application. Every choice in the stack is made to support offline-capable, local-network deployment inside a college placement cell — without dependency on web servers, cloud services, or browser-based frameworks.

Layer	Technology	Purpose
Programming Language	Java	Core application logic, business rules, and desktop runtime

Graphical Interface	Java Swing	Native desktop forms, tables, dialogs, and navigation for all roles
Database	MySQL	Relational persistence for students, companies, drives, applications, and reports
Database Connectivity	JDBC	Standard Java API connecting the application to MySQL
IDE	IntelliJ IDEA	Development, debugging, refactoring, and build management
Version Control	Git	Local source history, branching, and change tracking
Repository Hosting	GitHub	Remote repository for collaboration, backup, and academic submission

11.1 Technology Selection Rationale

Java was chosen for its platform independence — the same application runs on Windows, macOS, and Linux lab machines with a single JRE installation — together with a mature ecosystem, strong typing that maps naturally onto placement-domain entities such as Student, Company, Drive, and Application, and its position as a core part of the computer-science curriculum, which keeps the codebase approachable for evaluators and future maintainers.

Java Swing gives a native desktop experience suited to administrative data entry, with zero deployment overhead since it ships with the JRE and needs no web server or browser compatibility matrix. Its table, form, dialog, and tabbed-pane components map directly onto the searchable lists, review queues, and dashboards the placement workflow needs, and the whole interface runs offline on a local network — important in labs where internet access can be unreliable.

MySQL provides relational integrity through foreign keys, preventing the orphaned records that plague spreadsheet-based tracking, while its SQL engine supports the search, filter, report, and analytics requirements directly. It is familiar in academic lab environments, has proven scalability well beyond the projected 500 students and 50 drives, and its mysqldump tooling supports the backup requirements defined in Section 10.

JDBC removes any need for middleware between the Swing client and MySQL, gives explicit parameterized SQL control that both prevents injection and aids debugging, and supports the transactional commit and rollback semantics that keep multi-step writes atomic. A single driver JAR keeps the overall project lightweight. IntelliJ IDEA, Git, and GitHub round out the toolchain with integrated debugging, a built-in database console, branch-based feature development, and remote backup with academic-review access.

12. System Architecture

PlacePro follows a layered desktop architecture in which each tier has a single, well-defined responsibility. User interactions originate at the Java Swing presentation layer, pass through business logic for validation and workflow enforcement, descend to the JDBC data access layer for database operations, and terminate in the MySQL database where all persistent state resides. This separation of concerns is what keeps the codebase maintainable as the feature set grows.

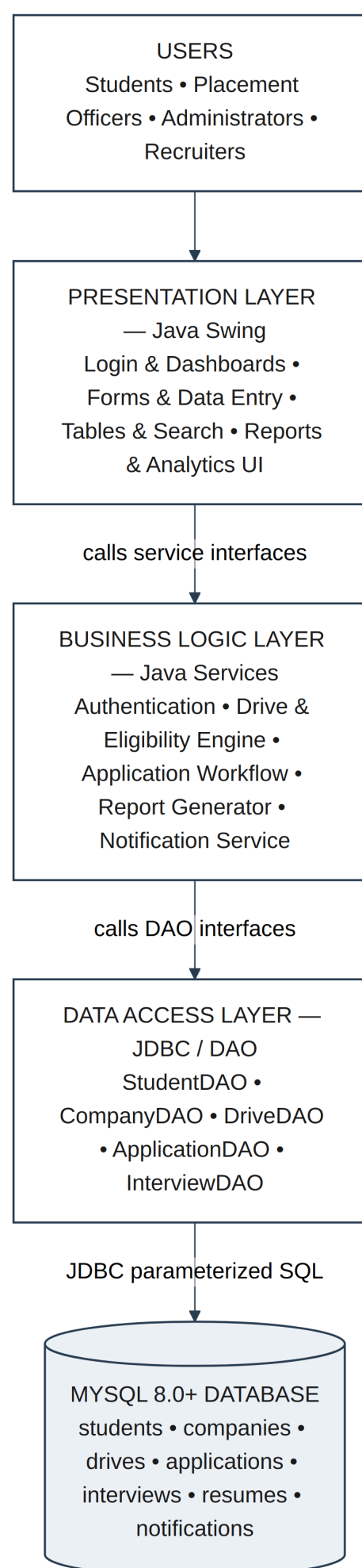


Figure 4 — Layered system architecture: presentation, business logic, data access, and database.

12.1 Boundary Rules

Four boundary rules keep the layers independent: Swing classes call service interfaces only and never import or instantiate DAO classes directly; service classes call DAO interfaces and never create JDBC Connection objects or write SQL strings; DAO classes contain SQL and JDBC code only, with no Swing imports and no business-rule logic such as eligibility checks; and the database stores data only — business logic lives in the service layer, not in stored procedures, for v1.0.

12.2 End-to-End Example — Student Applies to a Drive

A student selects a drive and clicks Apply. The Swing panel reads the student and drive identifiers and calls the application service, which first calls the eligibility service — if the student is ineligible, the request returns an error without ever touching the database. If eligible, the service checks for a duplicate application; if none exists, it builds an application record with status Applied and calls the DAO to insert it, together with a notification, inside a single transaction. Once the parameterized insert commits and the foreign-key constraints on student and drive have been validated, the Swing layer shows a confirmation dialog and the student immediately sees the new application listed with status Applied.

12.3 Cross-Cutting Concerns

Concern	How It Is Applied
Authentication	The authentication service validates credentials via the student and officer DAOs; the session token is held in memory.
Role-based access	Service methods check role before executing; Swing shows or hides menus according to role.
Input validation	The UI validates format; the service layer validates business rules such as eligibility and duplicated detection.

Error handling	DAO classes catch SQL exceptions; the service layer translates them into domain errors; Swing shows only user-friendly messages.
Logging	A logger writes authentication events, SQL errors, and unhandled exceptions to a log file.
Configuration	The database connection layer reads host, port, database, username, and password from a configuration file.
Resume files	The resume service writes the file to disk while the resume DAO stores only the metadata path in MySQL.

13. Database Design

PlacePro persists all business data in a MySQL 8.0+ relational database accessed via JDBC. The schema comprises nine tables normalized to Third Normal Form, using the InnoDB storage engine to support foreign keys and transactional integrity.

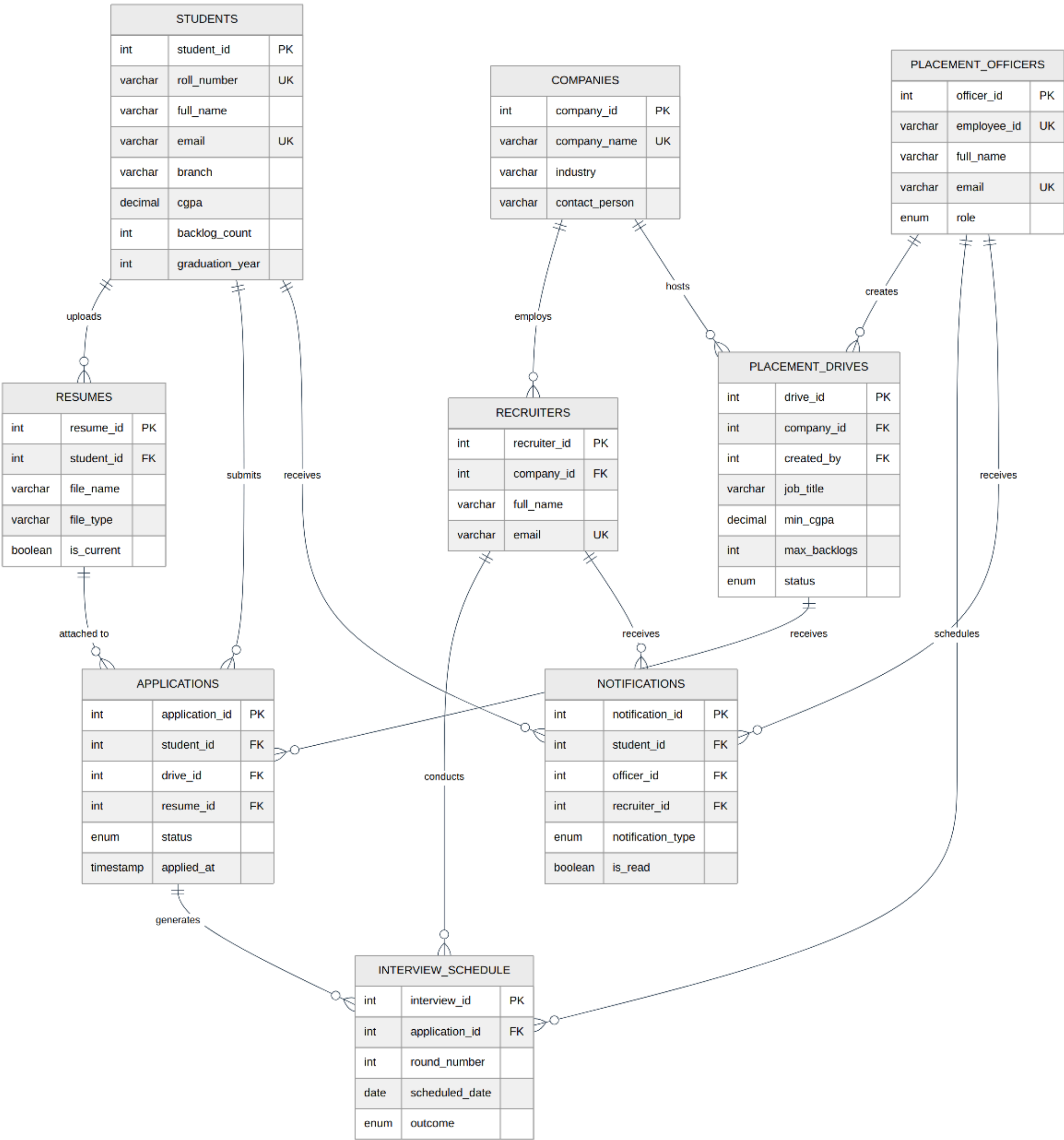


Figure 5—Entity-relationship diagram for the PlacePro database.

13.1 Schema Summary

#	Table	Purpose	Row Estimate (Season)
1	students	Student profile and academic eligibility data	~500

2	placement_officers	Officer and administrator accounts (role field distinguishes them)	~5– 10
3	recruiters	Recruiter accounts linked to a company	~30– 50
4	companies	Recruiting company directory	~30– 50
5	placement_drives	Drives linked to a company with eligibility rules and lifecycle state	~50
6	resumes	Resume file metadata linked to a student	~500– 1,000
7	applications	Student applications to drives, with status and history	~2,000– 5,000
8	interview_schedule	Interview rounds and outcomes linked to an application	~1,000– 3,000
9	notifications	Per-user in-app notifications	~5,000– 10,000

13.2 Normalization

All nine tables satisfy First, Second, and Third Normal Form: every column holds an atomic value, every non-key attribute depends on the whole primary key, and no attribute is transitively dependent on another non-key attribute — related entities are always referenced by foreign key rather than duplicated. One deliberate trade-off is documented: the `allowed_branches` column on `placement_drives` stores a comma-separated list of eligible branches rather than a fully normalized junction table, which keeps drive creation simple for v1.0 while leaving a clear extension path — a `drive_eligible_branches` table — if branch-level querying becomes necessary later. Referential integrity is enforced throughout: `recruiters` cascade from `companies`, `resumes` and `applications` cascade from `students`, and `interview` records cascade from `applications`, so that deleting a parent record cannot leave orphaned children.

14. Risk Assessment

This section identifies the principal risks associated with developing, deploying, and operating PlacePro. Likelihood is rated High, Medium, or Low based on how probable a risk is during normal operation; Impact is rated by the consequence if it occurs—from minor inconvenience to data loss or season disruption; Severity is derived from the two and drives mitigation priority.

14.1 Risk Register

ID	Risk	Likelihood	Severity	Mitigation
R1	MySQL server failure during an active placement season	Medium	High	Automated restart, daily mysqldump backups, connection retry with friendly error dialogs
R2	Data loss from absent or untested backups	Medium	Critical	Enforced pre-season full backup, tested restore procedure, restricted DROP privileges
R3	SQL injection via concatenated SQL in DAO classes	Low	High	Mandatory PreparedStatements for all queries; DAO-focused code review
R4	Database credentials hardcoded or committed to GitHub	Medium	High	Credentials in a .gitignore-excluded config file; rotate on exposure
R5	Passwords stored in plaintext or with weak hashing	Low	High	BCrypt hashing with work factor ≥ 10 ; never log or display plaintext
R6	Concurrent writes to the same record causing lost updates	Medium	Medium	InnoDB row-level locking, transactional JDBC, <code>updated_at</code> concurrency checks
R7	A user forgets to log out on a shared lab workstation	High	High	Explicit logout, session timeout, idle-state warning prompt
R8	Long-running queries freeze the Swing UI thread	Medium	Medium	Offload long operations to background worker threads with progress indicators
R9	Uploaded resume files inaccessible due to path or disk issues	Medium	Medium	Validate upload directory at startup; enforce file size limits; include in backups
R10	Insufficient role-based access enforcement allows privilege escalation	Low	High	Role checks in every service method; integration tests for unauthorized access

R11	Unclosed JDBC connections exhaust the connection pool	Medium	High	try-with-resources for all JDBC objects; centralized connection lifecycle
R12	Free-text branch entry causes inconsistent eligibility rules	Medium	Medium	Standardized branch dropdown instead of free text; documented conventions
R13	Single MySQL server is a point of failure for all clients	Medium	High	Maintenance scheduled outside placement season; tested rapid restore
R14	Insufficient input validation corrupts eligibility checks or reports	Medium	Medium	Field-level UI validation, service-layer business rules, MySQL column constraints

14.2 Severity Summary and Priority

Of the fourteen identified risks, one — the possibility of data loss from an absent or untested backup — is rated Critical and must be closed before the application is used in a live placement season. Eight risks are rated High, most of them concentrated around credential handling, database availability, and access control, and are addressed first: backup and restore procedures, externalized credentials with BCrypt hashing, PreparedStatements with try-with-resources, and role-based access-control tests. The remaining five Medium risks — concurrency, UI responsiveness, resume storage, branch-code consistency, and input validation — are accepted as manageable with the mitigations already listed, given the project's college-lab scale of roughly 500 students and 50 drives per season.

15. Future Enhancements

The items below extend PlacePro beyond its v1.0 desktop scope. They are organized into three tiers by how much architectural disruption each would introduce, and every tier depends on the layered architecture, normalized schema, and role-based access model described in this document remaining stable.

Tier	Enhancements
Tier 1—Near-term	Automated email notifications, a self-service company portal for recruiters, and deeper placement analytics — all low-disruption extensions of the existing notification and reporting foundations.
Tier 2—Mid-term	SMS notifications, a quantitative resume-to-role compatibility score, standardized student skill assessments, and a companion mobile application — each requiring an external integration or an additional client platform.
Tier 3—Long-term	AI-based resume analysis, built-in online coding tests, embedded video interviews, and migration to a cloud-hosted deployment — significant new subsystems or infrastructure changes best undertaken once the desktop platform is well established.

No enhancement in this roadmap is required for a demonstrable v1.0; each is deferred specifically because it would introduce either an external dependency (email, SMS, video) or a new client platform (mobile, cloud) that falls outside the local, offline-capable desktop scope defined in Section 5.4.